

Sección	Corrección	Sugerencia	Acción realizada
Metodología	· "Alta. Soporta cambios mediante refactorización continua" (Tabla 1.1) – Si bien es correcto, en XP la refactorización es una práctica continua, pero no es la única habilitante de cambios;	Cambiar a: "cambios mediante ciclos cortos y refactorización constante".	✅ Aceptada y corregida en el texto (Pág. 3)
Requerimientos	RNF-01: "completarse en menos de 3 min" sin especificar medición	Especificar método, herramientas (Lighthouse, Web Vitals), conexión de referencia (10 Mbps) y rango de medición	✅ Se borró este requerimiento porque no se reconoce una definición que no sea ambigua
Arquitectura	"Fácil de gestionar inicialmente" (monolito) → término vago	Cambiar a: "fácil de desplegar y depurar inicialmente"	✅ Aceptada y corregida (Pág. 14)
Arquitectura	Inconsistencia: "Mapper" vs "Data Mapper" sin especificar	Aclarar si se refiere al patrón de Fowler o a un DTO mapper	✅ Aceptada: se especifica como "Mapper de DTOs" (Pág. 19)
Arquitectura	El argumento "en un monolito las llamadas a funciones son instantáneas y predecibles" es correcto en teoría, pero en la práctica una aplicación web monolítica también sufre latencia de red entre cliente y servidor, y el overhead de serialización en la capa de presentación. La frase puede inducir a error si se interpreta como que no hay latencia alguna.	Aclarar que la latencia de red y serialización siguen existiendo en cliente-servidor	✅ Aceptada: se añadió matización (Pág. 15)
Arquitectura	Transición abrupta entre 1.4.3 y 1.5 (Clean → BD)	Añadir párrafo de enlace sobre modelo relacional como implementación de infraestructura	✅ Aceptada: se insertó el párrafo sugerido (Pág. 20)
Arquitectura	Falta de profundidad en patrones arquitectónicos (pág. 17): Se mencionan Repository, Unit of Work, Inyección de dependencias	Sugerencia lo puedes explicar aquí o dejarlo para el siguiente capítulo de detalles de	✅ Aceptada: se profundiza un poco más sobre los patrones pero se evita mencionar

Sección	Corrección	Sugerencia	Acción realizada
	y Mapper, pero no se explica cómo se implementan concretamente en el contexto de Clean Architecture (por ejemplo, si el Repository retorna entidades de dominio o modelos de persistencia). Tampoco se aclara si se utiliza un ORM (Entity Framework Core, NHibernate) o acceso directo.	implementación. Si te vas por esa opción entonces mencionas que en el capítulo 4 se explicará en más detalles	ORM, o implementaciones específicas. Se menciona la final sobre el Cap4. de implementaciones (Pág. 19-20)
Arquitectura	No se menciona el manejo de sesiones o transacciones distribuidas – Aunque el sistema es monolítico, el uso de Unit of Work sugiere transacciones con la base de datos. No se especifica el nivel de aislamiento ni cómo se manejan los conflictos de concurrencia	Especificar nivel de aislamiento (ej. Read Committed) y mecanismo de concurrencia (ej. optimistic con row version)	✅ Aceptada: de la misma forma que anterior, no se menciona. El uso de optimismo con RowVersion para la concurrencia no se menciona aquí sino en el siguiente Cap
Diseño BD	Inconsistencia: GUID en Evaluación vs enteros autoincrementales en otras tablas	Justificar: enteros para tablas transaccionales de alta frecuencia; GUID para futura integración	✅ Aceptada: se añadió justificación (Pág. 21)
Diseño BD	Seguridad: PBKDF2 con 10k iteraciones (obsoleto)	Aumentar a 600k iteraciones (NIST SP 800-132) o reconocer limitación por compatibilidad	✅ Aceptada: se tuvo en cuenta para el siguiente capítulo pero no se menciona en este. (Pág. 22)
Diseño BD	Sección 1.5.7 Normalización: dependencias funcionales en enumeración densa	Convertir en tabla de dos columnas: Determinante → Dependientes	✅ Aceptada: se quita al tabla y se justifica en Pag 35
Diseño BD	Código único de reporte: "algoritmo criptográficamente seguro" sin especificar	Especificar si es UUID v7, hash, o token; garantizar unicidad ante cortes de energía	✅ Aceptada: son detalles de implementación que se mencionan en el siguiente capítulo